
Assignment 04

Exercise 1:

Convert the following RISC-V assembly codes into machine language. Write the instructions in hexadecimal.

1. Codes 1:

```
1 addi s3, s4, 28
2 sll t1, t2, t3
3 srli s3, s1, 14
4 sw s9, 16(t4)
```

2. Codes 2:

```
1 add s7, s8, s9
2 srai t0, t1, 0xC
3 ori s3, s1, 0xABC
4 lw s4, 0x5C(t3)
```

3. Consider all instructions with an immediate field.

- Which instructions from codes use an immediate field in their machine code format?
- What is the instruction type (I-, S-, B-, U-, or J-type) for the instructions from part (a)?
- Write out the 5- to 21-bit immediates of each instruction from part (a) in hexadecimal. If the number is extended, also write them as 32-bit extended immediates. Otherwise, indicate that they are not extended.

Exercise 2:

Consider the RISC-V machine code snippet below. The first instruction is listed at the top.

- Convert the machine code snippet into RISC-V assembly language.
- Reverse engineer a high-level program that would compile into this assembly language routine and write it. Clearly comment your code.
- Explain in words what the program does. a0 and a1 are the inputs, and they initially contain positive numbers, A and B. At the end of the program, register a0 holds the output (i.e., return value).

```
1 0x01800513
2 0x00300593
3 0x00000393
4 0x00058E33
5 0x01C54863
6 0x00138393
7 0x00BE0E33
8 0xFF5FF06F
9 0x00038533
```

Exercise 3:

Repeat Exercise 2 for the following machine code. a0 and a1 are the inputs. a0 contains a 32-bit number and a1 is the address of a 32-element array of characters (char).

```

1 0x01F00393
2 0x00755E33
3 0x001E7E13
4 0x01C580A3
5 0x00158593
6 0xFFFF38393
7 0xFE03D6E3
8 0x00008067

```

Exercise 4:

Convert the following branch instructions into machine code. Instruction addresses are given to the left of each instruction.

1. Code 1:

```

1      0x0000A000 beq t4, zero, Loop
2      0x0000A004 ...
3      0x0000A008 ...
4      0x0000A00C Loop: ...
5

```

2. Code 2:

```

1      0x00801000 bne s5, a1, L1
2      ...
3      0x0080174C L1: ...
4

```

3. Code 3:

```

1      0x0000C10C Back: ...
2      ...
3      0x0000D000 blt s1, s2, Back
4

```

4. Code 4:

```

1      0x01030AAC bge t4, t6, L2
2      ...
3      0x01031AA4 L2: ...
4

```

5. Code 5:

```

1      0x0BC08004 L3: ...
2      ...
3      0x0BC09000 beq s3, s7, L3
4

```

Exercise 5:

Convert the following jump instructions into machine code. Instruction addresses are given to the left of each instruction.

1. Code 1:

```

1      0x1234ABC0 j Loop
2      ...
3      0x123CABBC Loop: ...
4

```

2. Code 2:

```

1      0x12345678 Back: ...
2      ... ...
3      0x123B8760 jal s0, Back
4

```

3. Code 3:

```

1      0xAABBCCD0 jal L1
2      ... ...
3      0xAABDCD98 L1: ...
4

```

4. Code 4:

```

1      0x11223344 j L2
2      ... ...
3      0x1127BCDC L2: ...
4

```

5. Code 5:

```

1      0x9876543C L3: ...
2      ... ...
3      0x9886543C jal L3
4

```

Exercise 6:

Consider the following RISC-V assembly language snippet. The numbers to the left of each instruction indicate the instruction address.

```

1 0xA0028 Func1: addi t4, a1, 0
2 0xA002C ori a0, a0, 32
3 0xA0030 sub a1, a1, a0
4 0xA0034 jal Func2
5 ... ...
6 0xA0058 Func2: lw t2, 4(a0)
7 0xA005C sw t2, 16(a1)
8 0xA0060 srli t3, t2, 8
9 0xA0064 beq t2, t3, Else
10 0xA0068 jr ra
11 0xA006C Else: addi a0, a0, 4
12 0xA0070 j Func2

```

1. Translate the instruction sequence into machine code. Write the machine code instructions in hexadecimal.
2. List the instruction type and addressing mode used at each line of code.

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

RV32I Base Instruction Set

imm[31:12]					rd	0110111	LUI
imm[31:12]					rd	0010111	AUIPC
imm[20 10:1 11 19:12]					rd	1101111	JAL
imm[11:0]			rs1	000	rd	1100111	JALR
imm[12 10:5]		rs2	rs1	000	imm[4:1 11]	1100011	BEQ
imm[12 10:5]		rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]		rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[12 10:5]		rs2	rs1	101	imm[4:1 11]	1100011	BGE
imm[12 10:5]		rs2	rs1	110	imm[4:1 11]	1100011	BLTU
imm[12 10:5]		rs2	rs1	111	imm[4:1 11]	1100011	BGEU
imm[11:0]			rs1	000	rd	0000011	LB
imm[11:0]			rs1	001	rd	0000011	LH
imm[11:0]			rs1	010	rd	0000011	LW
imm[11:0]			rs1	100	rd	0000011	LBU
imm[11:0]			rs1	101	rd	0000011	LHU
imm[11:5]		rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]		rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]		rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]			rs1	000	rd	0010011	ADDI
imm[11:0]			rs1	010	rd	0010011	SLTI
imm[11:0]			rs1	011	rd	0010011	SLTIU
imm[11:0]			rs1	100	rd	0010011	XORI
imm[11:0]			rs1	110	rd	0010011	ORI
imm[11:0]			rs1	111	rd	0010011	ANDI
0000000		shamt	rs1	001	rd	0010011	SLLI
0000000		shamt	rs1	101	rd	0010011	SRLI
0100000		shamt	rs1	101	rd	0010011	SRAI
0000000		rs2	rs1	000	rd	0110011	ADD
0100000		rs2	rs1	000	rd	0110011	SUB
0000000		rs2	rs1	001	rd	0110011	SLL
0000000		rs2	rs1	010	rd	0110011	SLT
0000000		rs2	rs1	011	rd	0110011	SLTU
0000000		rs2	rs1	100	rd	0110011	XOR
0000000		rs2	rs1	101	rd	0110011	SRL
0100000		rs2	rs1	101	rd	0110011	SRA
0000000		rs2	rs1	110	rd	0110011	OR
0000000		rs2	rs1	111	rd	0110011	AND
0000	pred	succ	00000	000	00000	0001111	FENCE
0000	0000	0000	00000	001	00000	0001111	FENCE.I
0000000000000			00000	000	00000	1110011	ECALL
0000000000001			00000	000	00000	1110011	EBREAK
csr			rs1	001	rd	1110011	CSR.RW
csr			rs1	010	rd	1110011	CSR.RS
csr			rs1	011	rd	1110011	CSR.RC
csr			zimm	101	rd	1110011	CSR.RWI
csr			zimm	110	rd	1110011	CSR.RSI
csr			zimm	111	rd	1110011	CSR.RCI